SPÉCIFICATION NORMATIVE · EXTRAIT DU CAHIER DES CHARGES

Moteur de rendez-vous récurrents

Spécification normative : bornes de répétition, exceptions, conflits, fuseaux horaires et changement d'heure

24	4	26	60
règles R-01 à R-24	modes de fin	exemples chiffrés	cas de test

Produit : MyRDV · **Document** : 06
Date : 12 novembre 2026 · **Statut** : normatif
Implémentation de référence : tools/recurrence_simulator.py

Confidentiel — diffusion restreinte

Sommaire

06 — Moteur de rendez-vous récurrents 5

1. Objet et promesse . . . 5
2. Modèle conceptuel . . . 5
3. Les bornes de répétition : quatre modes exclusifs . . . 6
   - 3.1 Sémantique exacte de AFTER_DURATION (le cas « 3 mois ») . . . 6
   - 3.2 Arithmétique des mois (addMonths) . . . 7
   - 3.3 NEVER et revalidation . . . 7
4. Table normative des règles . . . 7
5. Détail des règles . . . 8
   - R-01 — Fréquence hebdomadaire et jours multiples . . . 8
   - R-02 — Intervalle en semaines . . . 9
   - R-03 — Fin par nombre d'occurrences . . . 9
   - R-04 — Fin par date . . . 9
   - R-05 — Fin par durée . . . 9
   - R-06 — Heure murale et changements d'heure . . . 9
   - R-07 — Jours fériés, fermetures et congés . . . 10
   - R-08 — Exceptions exdate / rdate . . . 10
   - R-09 — Politique de conflit et budget . . . 11
   - R-10 — Plafonds de sécurité . . . 11
   - R-11 — Horizon de matérialisation glissant . . . 12
   - R-12 — Portée des modifications . . . 12
   - R-13 — Détachement d'une occurrence . . . 12
   - R-14 — Aucune perte silencieuse . . . 12
   - R-15 — Bornage des séries sans fin . . . 13
   - R-16 — Plafonds par offre . . . 13
   - R-17 — Idempotence et régénération . . . 13
   - R-18 — Notifications de série . . . 14
   - R-19 — Réservation des ressources par occurrence . . . 14
   - R-20 — Fuseau de référence et affichage . . . 14
   - R-21 — Annulation et liste d'attente . . . 15
   - R-22 — Suppression, annulation et cascade . . . 15
   - R-23 — Facturation des séries . . . 15
   - R-24 — Export ICS et interopérabilité . . . 16
6. Algorithme de génération (pseudo-code normatif) . . . 16
   - 6.1 Invariants vérifiés automatiquement . . . 16

6.2 Complexité et performance . . . 17
7. Cas limites et exemples numériques . . . 17
7.1 Détail E11 — traversée du changement d'heure (printemps) . . . 18
7.2 Détail E12 — ambiguïté d'automne . . . 18
7.3 Cas des semaines à cheval sur deux mois . . . 18
7.4 Cas d'un client en liste d'attente sur une série . . . 19
8. Interaction avec les autres modules . . . 19
9. Paramétrage et administration . . . 19
10. Erreurs et codes . . . 19
11. Critères d'acceptation du module . . . 20
12. Traçabilité . . . 21
13. Questions ouvertes (à arbitrer avant la fin du lot 1) . . . 21
14. Validation de cette spécification . . . 22

07 — API et schémas **23**

1. Conventions . . . 23
2. Ressources et endpoints . . . 23
2.1 Séries (14 endpoints) . . . 23
2.2 Occurrences (5 endpoints) . . . 24
2.3 Aides à la décision et ressources d'intégration (27 endpoints) . . . 24
3. Créer une série . . . 25
3.1 Requête . . . 25
3.2 Réponse 201 Created . . . 26
3.3 Code de réponse et sémantique . . . 26
4. Prévisualiser (dryRun) . . . 26
5. Machines à états . . . 27
5.1 États de la série . . . 27
5.2 États de l'occurrence . . . 27
5.3 Progression exposée . . . 28
6. Politique de conflit dans l'API . . . 29
7. Modifier une série . . . 29
7.1 Portées . . . 29
7.2 Requête ALL . . . 29
7.3 Réponse avec diff obligatoire . . . 29
7.4 Suppression . . . 30
8. Catalogue d'erreurs . . . 30
9. Webhooks . . . 31
10. Exemples d'intégration . . . 31
10.1 Créer une série bornée à 3 mois (cURL) . . . 31

10.2 Décaler une seule occurrence (Node.js) . . . 32
10.3 Gérer une création partielle (Python) . . . 32
10.4 Rejouer une requête en toute sécurité . . . 32
11. Atlas des champs de la ressource Series . . . 32
12. Conformité du contrat . . . 33

Document 06

Moteur de rendez-vous récurrents

Document : 06 / 11 · **Version** 1.0 · **Date** : 12 novembre 2026 · **Statut** : normatif **Portée** : règles de récurrence hebdomadaire, bornes de répétition, exceptions, conflits, ressources, matérialisation, notifications, facturation, interopérabilité. **Implémentation de référence** : `tools/recurrence_simulator.py` (specification exécutable, Python 3.9+, sans dépendance) — tout écart entre le code de production et ce simulateur est un **défaut**. Vecteurs de test : `annexes/vecteurs-test-recurrence.json`.

> **Note**
> **Primauté.** Ce document prévaut sur tous les autres en matière de récurrence. En cas de contradiction avec le document 03 (exigences) ou 07 (API), le présent document s'applique. Toute modification d'une règle `R-xx` est un **changement de contrat** : revue d'architecture + mise à jour du simulateur + du schéma JSON + des vecteurs de test, dans la même livraison.

1. Objet et promesse

Le module permet de poser, en une action, **une série de rendez-vous hebdomadaires bornée** : « une fois par semaine pendant 3 mois », « toutes les 2 semaines le mardi et le jeudi, 12 séances », « chaque lundi jusqu'au 15 décembre », ou « sans fin, à revalider tous les 3 mois ».

La promesse est triple :

Promesse	Traduction normative	Preuve
Prévisible : l'utilisateur sait exactement combien de séances et lesquelles	Prévisualisation obligatoire avant création (R-14), comptage exact, liste datée	Test T-REC-005
Fidèle : le calendrier ne dérive jamais	Heure murale conservée (R-06), arithmétique de mois explicite (R-05)	Vecteurs DST E8/E9/E10
Réversible : toute modification est ciblée et auditable	Portée *cette occurrence / les suivantes / la série* (R-12), détachement (R-13), idempotence (R-17)	Tests T-REC-021 à T-REC-039

Non-objectifs de la V1 : récurrence quotidienne (`FREQ=DAILY`), mensuelle par ordinal (« 3e mardi du mois »), annuelle, récurrence par « toutes les N semaines paires/impaires », séries multi-praticiens alternées. Ces besoins sont couverts par la V2 (doc 10 §2) ; le schéma JSON réserve les valeurs d'énumération sans les accepter.

2. Modèle conceptuel

```
Série (Series) ──1──n──> Occurrence (Appointment)
   │                          │
   ├── Règle (RecurrenceRule) : freq, interval, byWeekday, startDate, startTime, timezone
   ├── Fin (EndCondition)     : AFTER_COUNT | ON_DATE | AFTER_DURATION | NEVER
   ├── Exceptions             : exdate[], rdate[], détachements
   ├── Politiques             : conflictPolicy, dateShiftPolicy, buffers, ressources
   └── État                   : ACTIVE | ENDED | CANCELLED | EXHAUSTED, compteur 3/14
```

Terme	Définition
Série	Objet persistant portant la règle. Source de vérité. Ne contient pas les dates ; celles-ci sont **calculées**.
Occurrence	Rendez-vous concret issu d'une série. Porte `series_id` et une clé d'occurrence stable.
Date candidate	Date produite par la règle, avant application des exceptions et des conflits.
Exception `exdate`	Date candidate retirée définitivement (congé, accord du client).

Terme	Définition
Exception `rdate`	Date ajoutée hors règle (rattrapage, séance supplémentaire).
Occurrence détachée	Occurrence modifiée individuellement (heure, durée, salle) ; n'obéit plus à la règle mais reste rattachée à la série.
Horizon	Fenêtre de matérialisation des occurrences (90 jours par défaut, R-11).
Budget de conflits	Nombre maximal d'occurrences ignorées pour conflit avant refus (R-09).

Clé d'occurrence : `{series_id}@{YYYYMMDDTHHMM}` en heure locale de la série. Elle est **stable** (elle ne dépend pas de l'ordre de génération) et **unique** ; c'est la clé d'idempotence de la matérialisation (R-17) et de la synchronisation calendrier (R-24).

3. Les bornes de répétition : quatre modes exclusifs

L'exigence utilisateur (« 3 mois, une fois par semaine », « 12 séances », « jusqu'au 15 décembre ») se traduit par **une et une seule** condition de fin, choisie dans une liste fermée. Le mode est **exclusif** : envoyer `months` et `count` simultanément produit `SER_020_END_MODE_EXCLUSIVE`.

Mode	Paramètre	Sémantique	Cas d'usage
`AFTER_COUNT`	`count` (1..104)	Exactement N occurrences **planifiées** (les conflits ignorés ne comptent pas, R-09)	« 12 séances de kiné »
`ON_DATE`	`until` (date)	Toutes les occurrences dont la date est **≤** `until`, `until` incluse	« jusqu'au 15/12/2026 »
`AFTER_DURATION`	`months` (1..12) ou `weeks` (1..52)	Occurrences dont la date est **≤ startDate + durée** (bornes incluses)	« pendant 3 mois », « sur 6 semaines »
`NEVER`	—	Série sans fin, **bornée par les garde-fous** (R-10, R-15) et revalidée périodiquement	« suivi au long cours »

3.1 Sémantique exacte de AFTER_DURATION (le cas « 3 mois »)

Formellement : `until = addMonths(startDate, months)`, puis application du mode `ON_DATE`. La date de fin est **incluse**. Le nombre d'occurrences est donc `floor((until - startDate) / (7 × interval)) × |byWeekday| + |{jours de byWeekday dans la dernière semaine partielle}|`.

Exemple de référence de l'énoncé — début lundi 16/11/2026 à 09:00, un jour par semaine, intervalle 1, `months = 3` :

- `until = addMonths(2026-11-16, 3) = 2027-02-16` (inclus) ;
- occurrences : 16/11, 23/11, 30/11, 07/12, 14/12, 21/12, 28/12, 04/01, 11/01, 18/01, 25/01, 01/02, 08/02, 15/02 → **14 séances**, dernière le **15/02/2027**.

Les 13 semaines pleines du 16/11 au 15/02 donnent 13 séances, la 14e provient du fait que `2027-02-15` (lundi) est encore ≤ `2027-02-16`. C'est volontaire : l'utilisateur qui demande « 3 mois » obtient la période complète, pas 12 séances arbitrairement tronquées.

Table de référence par jour de départ (intervalle 1, 3 mois) :

Jour de départ	1re séance	Fin de période (`until`, incluse)	Nb séances	Dernière séance
lundi	2026-11-16	2027-02-16	14	2027-02-15
mardi	2026-11-17	2027-02-17	14	2027-02-16
mercredi	2026-11-18	2027-02-18	14	2027-02-17
jeudi	2026-11-19	2027-02-19	14	2027-02-18
vendredi	2026-11-20	2027-02-20	14	2027-02-19

Jour de départ	1re séance	Fin de période (`until` , incluse)	Nb séances	Dernière séance
samedi	2026-11-21	2027-02-21	14	2027-02-20
dimanche	2026-11-22	2027-02-22	14	2027-02-21

Autres durées, un jour par semaine, intervalle 1 (bornes incluses, donc `N+1` séances) :

Durée demandée	`until` (depuis 2026-11-16)	Nb séances
4 semaines	2026-12-14	5
6 semaines	2026-12-28	7
8 semaines	2027-01-11	9
12 semaines	2027-02-08	13
13 semaines	2027-02-15	14
26 semaines	2027-05-17	27
52 semaines	2027-11-15	53

> **Note**
> **Décision produit assumée (D2)** : la borne de fin est **incluse**. Ce choix est documenté dans l'interface par la mention « jusqu'au 15/02/2027 inclus » et par la prévisualisation du nombre exact de séances. L'alternative (borne exclue) a été écartée : elle produit des séries de 13 séances là où l'utilisateur en attend 14 et complique la reprise d'un suivi thérapeutique.

3.2 Arithmétique des mois (addMonths)

`addMonths(d, n)` conserve le jour du mois lorsque c'est possible, sinon **écrête** au dernier jour du mois cible. Aucun dépassement, aucune date invalide, jamais de « 31 février ».

Opération	Date source	+n	Résultat	Remarque
+1 mois	2026-01-31	1	2026-02-28	écrêtage
+1 mois	2024-01-31	1	2024-02-29	année bissextile
+3 mois	2026-11-30	3	2027-02-28	écrêtage
+6 mois	2026-08-31	6	2027-02-28	écrêtage
+12 mois	2024-02-29	12	2025-02-28	écrêtage
−3 mois	2026-05-31	−3	2026-02-28	symétrique

L'écrêtage **n'est pas cumulatif** : on calcule toujours à partir de la date de début de série, pas depuis la dernière occurrence calculée. Sinon, une série mensuelle démarrée un 31 glisserait progressivement vers le 28.

3.3 NEVER et revalidation

Une série `NEVER` n'est jamais « infinie » en base : elle est bornée par (R-10, R-15) `min(now + horizon, startDate + 12 mois,` plafond du plan `)` et **réévaluée à chaque réveil du planificateur** (quotidien). Une alerte est envoyée au praticien 30 jours avant la fin de la fenêtre de matérialisation (« votre suivi au long cours continue : 13 prochaines séances planifiées ») et la série peut être clôturée par l'utilisateur à tout moment.

4. Table normative des règles

Règle	Intitulé	Niveau	Exigences liées
R-01	Fréquence hebdomadaire et jours multiples	MUST	F-REC-01

Règle	Intitulé	Niveau	Exigences liées
R-02	Intervalle en semaines (1..8)	MUST	F-REC-02
R-03	Fin par nombre d'occurrences	MUST	F-REC-03, F-REC-04
R-04	Fin par date (incluse)	MUST	F-REC-03
R-05	Fin par durée (mois/semaines) et arithmétique de mois	MUST	F-REC-03, F-REC-04
R-06	Heure murale et changements d'heure	MUST	F-REC-18, F-TZ-03
R-07	Jours fériés, fermetures et congés	MUST	F-REC-07, F-AVL-03
R-08	Exceptions `exdate` / `rdate`	MUST	F-REC-08
R-09	Politique de conflit et budget de conflits	MUST	F-REC-06, F-APT-06
R-10	Plafonds de sécurité	MUST	F-REC-10
R-11	Horizon de matérialisation glissant	MUST	F-REC-15
R-12	Portée des modifications	MUST	F-REC-13
R-13	Détachement d'une occurrence	MUST	F-REC-09
R-14	Aucune perte silencieuse (rapport)	MUST	F-REC-17
R-15	Bornage des séries sans fin	MUST	F-REC-10, F-REC-16
R-16	Plafonds par offre et validation	MUST	F-ORG-08, F-REC-10
R-17	Idempotence et régénération	MUST	F-REC-15
R-18	Notifications de série	MUST	F-REC-12, F-NOT-10
R-19	Réservation des ressources par occurrence	MUST	F-RES-06
R-20	Fuseau de référence et affichage	MUST	F-TZ-01..04
R-21	Annulation et liste d'attente	MUST	F-WAIT-05
R-22	Suppression, annulation et cascade	MUST	F-REC-14
R-23	Facturation des séries	MUST	F-BIL-04, F-BIL-12
R-24	Export ICS / interopérabilité	SHOULD	F-REC-19, F-SYN-05

5. Détail des règles

R-01 — Fréquence hebdomadaire et jours multiples

Énoncé. Une série est définie par un ensemble **non vide et sans doublon** de 1 à 5 jours de la semaine (`byWeekday` , codage ISO-8601 : 1 = lundi … 7 = dimanche) et une fréquence `WEEKLY` . Les 7 jours sont refusés (une récurrence quotidienne déguisée en hebdomadaire nuit à la lisibilité de l'agenda et révèle une erreur de saisie : `SER_001_NO_WEEKDAY` pour 0 jour, `SER_002_BAD_WEEKDAY` pour un doublon ou une valeur hors 1..7).

Contrainte de cohérence. `startDate.weekday()` doit appartenir à `byWeekday` (`SER_005_START_WEEKDAY_MISMATCH`). L'interface ne propose que des dates cohérentes ; l'API refuse.

Ordre d'émission. À l'intérieur d'une même semaine, les occurrences sont émises par jour croissant (lundi avant mardi), jamais par ordre de saisie.

Exemple. `BYDAY=MO,WE,FR` , début lundi 16/11/2026, `months=3` → **40 séances** (3 jours × 13 semaines + 1), dernière le 15/02/2027.

R-02 — Intervalle en semaines

Énoncé. `interval` exprime le nombre de semaines entre deux semaines actives : `1` = toutes les semaines, `2` = une semaine sur deux, `3` = une sur trois, jusqu'à `8`. Les semaines non actives sont **entièrement sautées**, y compris tous leurs jours `byWeekday`.

Ancrage. Les semaines actives sont calculées depuis la **semaine ISO contenant la date de début**, et non depuis le début du mois. Changer `startDate` d'un jour dans la même semaine ne change donc pas la parité des semaines.

Contrainte. `1 ≤ interval ≤ 8` (`SER_003_BAD_INTERVAL`).

Exemple. `interval=2` , `BYDAY=MO` , début 16/11/2026, `months=3` → 16/11, 30/11, 14/12, 28/12, 11/01, 25/01, 08/02 → **7 séances** (la semaine du 23/11 et celle du 21/12, etc., sont sautées).

Exemple. `interval=2` , `BYDAY=TU` , `count=5` , début 17/11/2026 → 17/11, 01/12, 15/12, 29/12, 12/01/2027.

R-03 — Fin par nombre d'occurrences

Énoncé. `AFTER_COUNT` produit exactement `count` occurrences **planifiées** (statut `SCHEDULED`). Les occurrences ignorées pour cause de conflit (`SKIP`) ou de jour fermé (`SKIP`) **ne consomment pas** le compteur : l'utilisateur qui demande 12 séances en obtient 12 dans l'agenda, quitte à ce que la dernière tombe après la fenêtre initialement imaginée.

Borne. `1 ≤ count ≤ min(104, plafond de l'offre)` (`SER_006_BAD_COUNT` , `SER_007_COUNT_OVER_CAP`).

Garde-fou. Si le compteur n'est pas atteint à l'horizon de génération (`SAFETY_BOUND_12M` ou fin des candidats), la réponse est **partielle mais explicite** : `count` réel, `warnings[]` , `status = PARTIAL` ; l'interface affiche « 9 séances planifiées sur 12 demandées — 3 non plaçables ».

Exemple. `count=4` , `BYDAY=MO` , début 16/11/2026, conflit le 23/11 en politique `SKIP` → 16/11, **30/11**, 07/12, 14/12 (le 23/11 est ignoré, le compteur reste à 4).

R-04 — Fin par date

Énoncé. `ON_DATE` produit toutes les occurrences de date `≤ until` , `until` **incluse**. `until ≥ startDate` (`SER_008_BAD_UNTIL`) ; `until` exprimée en date locale de la série, jamais en UTC.

Exemple. `BYDAY=MO` , `until=2026-12-14` → 16/11, 23/11, 30/11, 07/12, **14/12** = 5 séances.

Interaction avec l'heure. Une occurrence du 14/12 à 09:00 est incluse même si l'utilisateur a créé la série le 14/12 à 10:00 — mais le garde-fou de délai minimum (R-10, 2 h) s'applique : une occurrence ne peut pas être planifiée à moins de 2 h de l'instant présent.

R-05 — Fin par durée

Énoncé. `AFTER_DURATION` accepte `months` (1..12) **ou** `weeks` (1..52), exclusivement (`SER_009_BAD_DURATION_END` , `SER_010_DURATION_OVER_CAP`). La borne est calculée par `addMonths(startDate, months)` ou `startDate + 7 × weeks jours`, puis traitée exactement comme `ON_DATE` .

Détail des mois. Voir §3.2 : écrêtage au dernier jour du mois, calcul toujours depuis `startDate` , jamais cumulatif.

Exemple. Début 30/11/2026 (lundi), `months=1` → `until = 2026-12-30` → 30/11, 07/12, 14/12, 21/12, 28/12 = **5 séances** (le 04/01 est hors borne).

Exemple. Début 16/11/2026, `weeks=6` → `until = 2026-12-28` → **7 séances**.

R-06 — Heure murale et changements d'heure

Énoncé. Une occurrence est définie par une **heure murale locale** dans le fuseau de la série (`09:00`), pas par un instant UTC. Le passage à l'heure d'été/hiver ne décale donc **jamais** l'heure du rendez-vous : un rendez-vous de 09:00 reste à 09:00, l'instant UTC se décale d'une heure.

Cas 1 — heure inexistante (passage à l'heure d'été, trou). `02:30` n'existe pas le 2027-03-28 à Paris. Résolution : **première heure valide ≥ l'heure demandée**, soit `03:00` (politique `SHIFT_TO_FIRST_VALID`). L'occurrence reste dans le même jour.

Cas 2 — heure ambiguë (passage à l'heure d'hiver, doublon). `02:30` existe deux fois le 2026-10-25 à Paris. Résolution : **premier passage** (`fold=0`, heure d'été), politique `EARLIER`.

Cas 3 — décalage lointain. Série de 09:00 à Montréal traversant le passage du 14/03/2027 : 09:00 EST (14:00 UTC) puis 09:00 EDT (13:00 UTC). L'heure murale est conservée, l'instant UTC change — c'est le comportement attendu par les utilisateurs et celui de RFC 5545.

Interdiction. Il est interdit de générer les occurrences en UTC puis de les convertir (erreur classique) : cela produirait des rendez-vous à 08:00 ou 10:00 deux fois par an. La génération part toujours de la date locale + l'heure murale, puis résout l'instant (`resolve_local` du simulateur).

Cas particulier des séries `count` : le compteur compte les occurrences **émises**, y compris celle décalée de 02:30 à 03:00.

R-07 — Jours fériés, fermetures et congés

Énoncé. Une série peut consommer un **calendrier de jours non travaillés** : jours fériés légaux du pays/region du lieu (calculés, y compris Pâques et les fêtes mobiles), fermetures de l'organisation, congés du praticien, indisponibilités ponctuelles, fermetures de salle.

Pour chaque date candidate tombant un jour non travaillé, la politique `dateShiftPolicy` s'applique :

Politique	Comportement	Cas d'usage
`SKIP` (défaut)	L'occurrence n'est pas planifiée ; elle est listée dans `skipped[]` avec motif	Fermeture annuelle, on ne rattrape pas
`SHIFT_FORWARD`	Report au **premier jour disponible suivant** (non férié, non fermé, hors `exdate`, non déjà occupé par la série), sans limite de jours mais avec un garde-fou de 60 jours	« Si c'est férié, on décale au lendemain »
`KEEP`	L'occurrence est maintenue malgré le férié (activité ouverte ce jour-là)	Clinique ouverte le 1er mai

Ordre d'application. R-07 s'applique **avant** R-09 (conflits) : un report de férié peut créer un conflit qui sera traité par la politique de conflit.

Interdiction des doublons. Le report ne doit jamais produire deux occurrences le même jour : si le jour suivant est déjà occupé par une autre occurrence de la série, on continue de décaler.

Exemple. `BYDAY=MO`, début 21/12/2026, `count=3`, `2026-12-28` déclaré fermé, `SHIFT_FORWARD` → 21/12, **29/12** (mardi), 04/01/2027.

Exemple. Même série en `SKIP` → 21/12, 04/01, 11/01 ; `skipped: [{2026-12-28, BLACKOUT}]`.

R-08 — Exceptions exdate / rdate

Énoncé. Deux listes de dates enrichissent la règle sans la modifier : `exdate` (retrait ; max 50 par série) et `rdate` (ajout ; max 20 par série). Elles sont idempotentes et ordonnées.

Contraintes.

- Une date ne peut être à la fois `exdate` et `rdate` (`SER_021_EXDATE_RDATE_CONFLICT`).
- `rdate` doit être ≥ `startDate` et ≤ borne de fin ; une `rdate` hors borne est ignorée avec avertissement.
- `rdate` ne consomme pas le compteur `AFTER_COUNT` (ce sont des séances supplémentaires, explicites).
- Une `exdate` ne consomme pas le compteur `AFTER_COUNT` (même logique que R-03).
- Les `exdate` restent visibles dans l'historique de la série (« 07/12 retiré par Camille le 30/11 »).

Interaction. `exdate` s'applique avant R-07 : une date retirée ne sera jamais reportée.

R-09 — Politique de conflit et budget

Énoncé. Pour chaque occurrence candidate, le moteur vérifie la disponibilité au sens large :

1. le praticien n'a pas d'autre rendez-vous chevauchant (tampons inclus) ;
2. la salle et les équipements requis sont libres ;
3. le créneau est dans une plage de disponibilité (semaine type ou plage exceptionnelle) ;
4. les plafonds de charge (rendez-vous/jour, rendez-vous/créneau) ne sont pas dépassés ;
5. le client n'a pas déjà un rendez-vous chevauchant ;
6. le délai minimal de réservation est respecté.

Si l'une de ces conditions échoue, `conflictPolicy` détermine la conduite :

Politique	Comportement	Échec
`FAIL`	**Tout ou rien** : aucune occurrence n'est créée si une seule est en conflit	`SER_017_SLOT_CONFLICT`
`SKIP` (défaut)	L'occurrence est ignorée, listée dans `skippedConflict[]`, la génération continue	`SER_018_CONFLICT_BUDGET` si budget dépassé
`SHIFT_SAME_DAY`	Recherche d'un créneau le **même jour** à ±15, ±30, … jusqu'à `shiftToleranceMinutes` (240 par défaut), d'abord plus tard puis plus tôt	`SER_017_SLOT_CONFLICT` si aucun créneau trouvé dans la tolérance, ou `SKIP` implicite si `shiftToleranceMinutes = 0`

Budget de conflits. En `SKIP`, au-delà de `conflict_skip_budget` (5 par défaut, 10 max), la création est refusée : au-delà, l'utilisateur doit revoir sa règle (autre jour, autre heure, autre praticien). Garde-fou complémentaire : **3 conflits consécutifs** → `SER_019_CONSECUTIVE_CONFLICTS` (cela signe une fermeture de période ou une erreur de saisie).

Atomicité. La génération s'exécute dans une transaction ; en cas d'échec, aucune occurrence n'est persistée (pas de série partielle cachée).

Prévisualisation. Le `dryRun` renvoie exactement le même rapport sans écrire (doc 07 §6).

Exemple (`SKIP`). 4 lundis à 09:00, le 23/11 occupé → occurrences 16/11, 30/11, 07/12, 14/12 ; `skippedConflict: [2026-11-23]`.

Exemple (`SHIFT_SAME_DAY`). Lundi 16/11 à 09:00 occupé jusqu'à 10:00 → occurrence placée à **10:00** (`shiftedMinutes: 60`), les suivantes restent à 09:00.

R-10 — Plafonds de sécurité

Plafond	Valeur	Motif
Occurrences par série	**104**	2 ans de séances hebdomadaires ; protège l'agenda et la base
Horizon absolu	**12 mois** à partir de `startDate`	au-delà, une règle n'a plus de sens prévisionnel
Intervalle maximal	**8 semaines**	au-delà, l'utilisateur devrait créer des rendez-vous isolés
Durée maximale `AFTER_DURATION`	12 mois / 52 semaines	cohérence avec l'horizon
Jours par série	**5**	7 = quotidien déguisé
`rdate` / `exdate`	20 / 50	protection contre l'usage détourné de l'API
Budget de conflits	5 (10 max)	anti-boucle
Délai minimum avant la 1re occurrence	**2 h**	`SER_016_LEAD_TIME`
Taille de réponse (`dryRun`)	400 occurrences max	pagination au-delà

Chaque dépassement produit un code d'erreur explicite (doc 07 §8), jamais une troncature silencieuse.

R-11 — Horizon de matérialisation glissant

Énoncé. Les occurrences sont matérialisées **à la demande** (lazy), à horizon glissant :

- horizon par défaut : **90 jours** (offre Pro), 30 jours (Solo), 180 jours (Business) ;
- un job quotidien (`series-materializer`) matérialise les occurrences manquantes des séries `ACTIVE` jusqu'à `now + horizon` ;
- les occurrences passées ne sont jamais supprimées ni régénérées (immuabilité de l'historique) ;
- une lecture d'agenda au-delà de l'horizon déclenche une matérialisation synchrone bornée à 400 occurrences (`sync_horizon_cap`).

Justification (D1). Matérialiser 12 mois d'avance rend chaque modification de règle coûteuse et provoque des incohérences au changement d'heure. L'horizon glissant garantit que toute correction de règle s'applique au futur proche, sans réécrire le passé.

Idempotence. Le job est rejouable : il ne crée que les clés d'occurrence absentes (R-17).

Observabilité. Métriques : occurrences matérialisées/min, retard du job (`lag`), séries en échec.

R-12 — Portée des modifications

Toute modification d'une occurrence d'une série précise sa **portée** :

Portée	Effet sur la règle	Effet sur les occurrences	Cas d'usage
`THIS`	Inchangée	Seule l'occurrence ciblée est détachée et modifiée	« Le 7/12 je suis absent, on décale à 14:00 »
`THIS_AND_FOLLOWING`	Nouvelle règle dérivée à partir de la date ciblée (`split`)	Les occurrences ≥ date sont supprimées puis régénérées par la nouvelle règle ; les antérieures restent	« À partir de janvier, on passe à toutes les 2 semaines »
`ALL`	Règle modifiée	Toutes les occurrences futures non détachées sont régénérées ; les détachées sont conservées	« Finalement, 09:30 au lieu de 09:00 »

Interdiction. La portée `ALL` ne modifie jamais une occurrence passée ni une occurrence détachée : elle produit un `diff` explicite (créées / modifiées / supprimées) présenté à l'utilisateur avant application (interface) ou renvoyé dans la réponse (API).

Réversibilité. Chaque application stocke un instantané de la règle précédente (`series_revision`) pendant 30 jours, permettant un retour arrière (`POST /series/{id}/revert`).

R-13 — Détachement d'une occurrence

Énoncé. Une occurrence peut être modifiée individuellement (heure, durée, lieu, salle, praticien, notes). Elle devient `detached = true` , conserve `series_id` et sa clé d'origine, et **n'est plus régénérée** par la règle : supprimer la série et la recréer ne la ressuscite pas (elle porte `detached_at`).

Ce qui reste interdit. On ne peut pas détacher une occurrence pour la déplacer sur une date qui appartient déjà à la série (`SER_022_OCCURRENCE_COLLISION`).

Affichage. Dans l'agenda, une occurrence détachée porte un indicateur visuel (« modifiée ») et un lien vers la série.

R-14 — Aucune perte silencieuse

Énoncé. Toute occurrence candidate non planifiée doit être expliquée. Le rapport de génération contient toujours :

```
{
  "count": 10,
  "requested": 12,
  "status": "PARTIAL",
  "skipped": [
    {"date": "2026-12-25", "reason": "BLACKOUT", "policy": "SKIP"},
    {"date": "2027-01-01", "reason": "BLACKOUT", "policy": "SKIP"}
  ],
  "skippedConflict": [
    {"date": "2026-11-23", "reason": "PRACTITIONER_BUSY", "detail": "Rendez-vous 09:00-10:00"}
  ],
  "warnings": ["2 séances non planifiées sur 12 demandées"]
}
```

Motifs normalisés : `BLACKOUT` (jour férié/fermeture), `LEAVE` (congé praticien), `PRACTITIONER_BUSY`, `RESOURCE_BUSY`, `CLIENT_BUSY`, `OUTSIDE_AVAILABILITY`, `DAILY_CAP`, `LEAD_TIME`, `EXDATE`, `HORIZON`.

Obligation d'interface. L'écran de résultat affiche un bandeau durable (non un toast de 3 s) tant que `skipped.length + skippedConflict.length > 0`, avec un lien vers le détail. L'API expose `GET /series/{id}/report`.

R-15 — Bornage des séries sans fin

Énoncé. Toute série `NEVER` est bornée par `min(now + horizon, startDate + 12 mois, maxOccurrences de l'offre)`. Son état expose `windowEndsAt`. Un réveil quotidien prolonge la fenêtre. Une série sans fin dont la fenêtre ne peut plus être prolongée (plafond de 104 atteint) passe à `EXHAUSTED` et notifie le praticien. `SER_015_NEVER_WITHOUT_CAP` protège l'API contre une demande explicite de génération infinie sans plafond.

R-16 — Plafonds par offre

Paramètre	Solo	Pro	Business	Entreprise
Occurrences max / série	30	104	104	104
Horizon de matérialisation	30 j	90 j	180 j	180 j
Intervalle max	4 sem.	8 sem.	8 sem.	8 sem.
Durée max `AFTER_DURATION`	3 mois	12 mois	12 mois	12 mois
Jours par série	3	5	5	5
Budget de conflits	3	5	10	10
`rdate` / `exdate`	5/10	20/50	20/50	20/50
Séries actives simultanées	20	200	1 000	illimité

Les plafonds sont **lus en configuration** (table `plan_feature`), jamais codés en dur dans le moteur ; seule la borne absolue R-10 est immuable. Un dépassement renvoie l'erreur `SER_007_COUNT_OVER_CAP` ou `SER_010_DURATION_OVER_CAP` avec, dans `details`, `limit`, `requested`, `plan` et un lien d'upgrade (l'interface affiche la comparaison des offres).

R-17 — Idempotence et régénération

Énoncé. La matérialisation est une fonction **pure et idempotente** de l'entrée `(règle, exceptions, politiques, fuseau, busy set, now)`. La clé d'occurrence (`series_id@YYYYMMDDTHHMM`) garantit :

- rejouer la génération ne crée **aucun doublon** ;
- régénérer après une modification ne recrée **jamais** une occurrence supprimée manuellement (`exdate` ou suppression individuelle = exception enregistrée) ;
- une occurrence `COMPLETED`, `NO_SHOW` ou facturée n'est jamais supprimée par une régénération.

Contrainte d'implémentation. La génération et l'écriture se font dans une transaction avec `INSERT ... ON CONFLICT (series_id, occurrence_key) DO NOTHING`, puis les occurrences orphelines (présentes en base, absentes du calcul, futures, non détachées) sont supprimées en fin de transaction. Aucune fenêtre de temps où les deux

versions coexistent.

Paramètre `now`. `now` est injecté (jamais `now()` implicite dans le moteur) afin de rendre les tests déterministes et de pouvoir rejouer une génération à une date donnée.

R-18 — Notifications de série

Moment	Destinataire	Canal	Contenu
Création de série	Client	courriel + SMS si activé	**Récapitulatif unique** : liste des dates planifiées, lien de gestion, politique d'annulation
Modification de portée `THIS`	Client	courriel/SMS	« Votre rendez-vous du 07/12 est déplacé au 07/12 14:00 »
Modification de portée `ALL` / `THIS_AND_FOLLOWING`	Client	courriel	Nouveau récapitulatif, avec mention des changements
Suppression d'occurrence	Client	courriel/SMS	Annulation avec motif et lien de reprise
7 jours avant le début de la série	Client	courriel	Rappel de la première séance
Chaque occurrence	Client	selon politique de rappels (F-NOT-02)	Rappel normal, **sans révélateur de série**
Fin de fenêtre (`NEVER`)	Praticien	notification interne	« 3 mois de planification restants »
Échec de régénération	Praticien + admin	notification interne + e-mail	Motif, série, action attendue

Règles de fond. Le récapitulatif est **un seul message** (pas 14 messages à la création) ; il respecte la langue du client (R-18, F-NOT-04) et **ne contient aucune donnée de santé** (NFR-CMP-06). Un client peut refuser les rappels mais reste destinataire des messages transactionnels essentiels (création, annulation), conformément à la base légale contractuelle.

Anti-flood. Une modification en masse (portée `ALL` sur une série de 40 séances) envoie **un** message de synthèse, jamais 40.

R-19 — Réservation des ressources par occurrence

Énoncé. Pour chaque occurrence, le moteur réserve de façon atomique : praticien, salle, équipements, créneau. Un conflit de ressource est traité exactement comme un conflit de praticien (R-09). Si une salle est occupée une seule fois sur 14 occurrences, la politique `SKIP` retire **uniquement** cette occurrence ; `FAIL` refuse la série entière ; `SHIFT_SAME_DAY` cherche une autre salle disponible à la même heure si le service le permet (`room_flexible = true`), puis un autre créneau dans la tolérance.

Cohérence. La ressource réservée est **la même** pour toutes les occurrences par défaut ; si l'utilisateur choisit « salle au choix », chaque occurrence peut recevoir une salle différente (respectant capacité et équipements).

Exemple d'échec attendu. Série de 12 lundis, salle 2 occupée le 21/12 et le 28/12, politique `FAIL` → `409` `SER_017_SLOT_CONFLICT` avec `details.dates = [2026-12-21, 2026-12-28]` ; politique `SKIP` → 10 occurrences + `skipped` explicite.

R-20 — Fuseau de référence et affichage

Énoncé. La série porte un fuseau IANA (celui du lieu, par défaut celui du praticien). Toutes les dates de la règle sont interprétées dans ce fuseau. L'API manipule :

- `startDate` / `until` : dates **locales** (`YYYY-MM-DD`), sans heure ;
- `startTime` : heure murale locale (`HH:MM`) ;
- chaque occurrence : `localStart` (avec offset), `utcStart` , `timezone` .

Affichage. Un utilisateur dans un autre fuseau voit l'heure dans **son** fuseau, avec l'heure de référence en second plan (« 09:00 heure de Paris »). Les durées ne sont jamais affectées par DST (une séance de 60 min reste 60 min

; si elle traverse un changement d'heure, elle dure 60 min réelles et l'heure de fin murale peut afficher 10:00 ou 11:00 selon le sens — comportement documenté dans l'interface par une infobulle).

Interdiction. Aucune série ne peut être stockée sans fuseau (`SER_023_TIMEZONE_REQUIRED`).

R-21 — Annulation et liste d'attente

Énoncé. L'annulation d'une occurrence (ou d'une série entière) libère le créneau au profit de la liste d'attente, avec deux règles spécifiques :

1. **Un créneau libéré par une série n'est jamais re-réservé automatiquement par la même série** : le moteur ne « rattrape » pas une occurrence annulée sur un autre créneau du même jour sans action explicite (sinon une annulation pourrait créer un déplacement non voulu).
2. **La liste d'attente est prioritaire sur la régénération** : lors d'une régénération, les créneaux déjà promis à la liste d'attente sont considérés comme occupés (R-09).

Séries et liste d'attente. Un client inscrit en liste d'attente pour « série de suivi » peut recevoir une proposition de série complète si le praticien ouvre un créneau récurrent dédié (`waitlist_series_offer` , V1.1).

Notification. L'annulation d'une occurrence déclenche `appointment.cancelled` ; le service de liste d'attente décide de la proposition, jamais le moteur de récurrence (découplage par événement).

R-22 — Suppression, annulation et cascade

Action	Effet sur les occurrences futures	Effet sur les passées	Effet sur les dépendances
`DELETE /series/{id}` (série entière)	Toutes supprimées (sauf facturées, qui passent à `CANCELLED`)	Conservées	Factures émises conservées ; documents détachés conservés
`DELETE ...?scope=THIS_AND_FOLLOWING&from=date`	Supprimées à partir de la date	Conservées	Factures futures annulées si non payées
`DELETE ...?scope=THIS&occurrence=key`	Une seule supprimée, enregistrée en `exdate`	—	Aucun impact
Suppression du praticien	Séries transférées ou annulées selon choix explicite ; jamais de suppression silencieuse	Conservées	Réaffectation proposée
Suppression du client	Séries annulées, occurrences passées anonymisées	Anonymisées	Factures conservées 10 ans

Confirmation. L'interface exige la saisie du nombre d'occurrences concernées (« supprimer 11 séances ? ») ; l'API exige `confirmCount` égal au nombre réel (`SER_024_CONFIRMATION_REQUIRED`).

Journalisation. Toute suppression de série est auditée (acteur, portée, nombre, avant/après).

R-23 — Facturation des séries

Énoncé. Une série est un **conteneur de rendez-vous**, pas une ligne de facture unique, sauf si le praticien choisit un forfait.

Mode	Débit	Cas d'usage
`PER_SESSION` (défaut)	Une facture par occurrence terminée	Ostéopathie, psychologie
`PREPAID_PACKAGE`	Une facture à la création pour N séances ; chaque occurrence consomme un crédit	Forfait 5 massages
`SUBSCRIPTION`	Prélèvement récurrent mensuel, indépendant des occurrences	Coaching

Règles de cohérence.

- Une occurrence annulée avant le délai autorisé ne génère pas de facture ; après le délai, la politique de frais s'applique (F-APT-03).

- Une occurrence `NO_SHOW` facturée reste facturée même si la série est supprimée ensuite (R-22).
- Le total d'un forfait est recalculé si la série est modifiée ; un avoir est émis si le nombre de séances baisse (`F-BIL-12`).
- Les remises de série sont possibles (montant fixe ou pourcentage) et tracées.

Exemple. Série de 12 séances à 60 €, forfait prépayé 660 € (−8 %) → 1 facture de 660 €, 12 crédits, avoir de 55 € si la série est ramenée à 11 séances après annulation d'une occurrence hors délai.

R-24 — Export ICS et interopérabilité

Énoncé. Une série s'exporte en **un seul** composant `VEVENT` portant `RRULE` (`FREQ=WEEKLY;INTERVAL=n;BYDAY=…`), `RDATE` / `EXDATE` , `DTSTART;TZID=…` . Les occurrences détachées sont exportées comme des `VEVENT` séparés portant `RECURRENCE-ID` .

Sémantique. Un client qui importe la série dans Google Agenda, Outlook ou Apple Calendrier doit voir les mêmes dates. Le `VALARM` est exporté pour le rappel. `UNTIL` est exprimé en UTC (`…Z`) conformément à RFC 5545, tandis que `COUNT` est préféré quand le mode est `AFTER_COUNT` .

Limites. `NEVER` est exporté avec `UNTIL` = fin de fenêtre ; à chaque prolongation, le flux (`webcal`) est mis à jour ; un fichier `.ics` téléchargé n'est pas mis à jour et l'interface le signale.

6. Algorithme de génération (pseudo-code normatif)

```
FONCTION genererSerie(série, busy, maintenant):
    VALIDER série                                    # R-01, R-02, R-03/04/05, R-10, R-16
    fin ← borneEffective(série, maintenant)          # min(règle, horizon, garde-fous) — R-03/04/05/11/15
    si premiereOccurrence(série) < maintenant + 2h   # R-08 / SER_016
        LEVER SER_016_LEAD_TIME
    dates ← datesCandidates(série, fin)              # R-01, R-02, ancrage semaine ISO
    dates ← union(dates, série.rdate)                 # R-08
    dates ← dates \ série.exdate                     # R-08
    dates ← appliquerJoursFermés(dates, série)       # R-07 → SKIP | SHIFT_FORWARD | KEEP
    trier(dates)
    occurrences ← []
    pour chaque date dans dates:
        si date > fin: continuer
        localStart ← résoudreHeureMurale(date, série.heure, série.fuseau)   # R-06
        si conflit(localStart, série, busy):                                # R-09
            si politique = FAIL: LEVER SER_017_SLOT_CONFLICT
            si politique = SKIP: skippedConflict.ajouter(date); continuer
            si politique = SHIFT_SAME_DAY:
                localStart ← chercherCreneauMemeJour(localStart, tolérance)
                si absent: skippedConflict.ajouter(date); continuer
        réserverRessources(localStart)                                      # R-19, transactionnel
        occurrences.ajouter(cle(date), localStart, localStart + durée, statut=SCHEDULED)
        si mode = AFTER_COUNT et |occurrences| = count: sortir                 # R-03
    ASSERT ordre chronologique, unicité des clés, aucune perte silencieuse     # R-14, R-17
    RETOURNER rapport(occurrences, skipped, skippedConflict, warnings, fin)
```

6.1 Invariants vérifiés automatiquement

#	Invariant	Vérification
I-1	Les occurrences sont strictement croissantes	test + assertion runtime
I-2	Les clés d'occurrence sont uniques	contrainte DDL
I-3	Chaque occurrence est ≤ borne de fin	assertion
I-4	Mode `AFTER_COUNT` ⇒ `|occurrences| ≤ count`	assertion
I-5	L'écart entre deux semaines actives est un multiple de `interval`	test
I-6	La génération est idempotente (même entrée ⇒ même sortie)	test de double exécution

#	Invariant	Vérification
I-7	Aucune occurrence n'est créée sans ressource libre	contrainte transactionnelle
I-8	Toute occurrence non planifiée est dans un rapport	contrat d'API
I-9	L'heure murale est identique d'une occurrence à l'autre, hors détachement	test DST
I-10	Aucune donnée n'est écrite si la transaction échoue	test d'atomicité

6.2 Complexité et performance

Opération	Complexité	Budget
Génération d'une série de 104 occurrences	O(n × c) avec c = coût de vérification de disponibilité (index B-tree)	**< 150 ms p95** sans `dryRun`, < 60 ms en `dryRun`
Prévisualisation (dryRun, 40 occurrences)	idem, sans écriture	< 80 ms p95
Régénération glissante (job quotidien, 10 000 séries)	parallélisable par partition `series_id`	< 10 min
Lecture d'agenda (semaine, 12 praticiens)	requête indexée	< 250 ms p95

La vérification de disponibilité s'appuie sur un index partiel `appointment(practitioner_id, range)` (type `tstzrange`) avec exclusion par `GIST` : la détection de conflit est faite **par la base de données** (`EXCLUDE USING GIST`), pas par le code applicatif, ce qui rend les collisions impossibles même sous concurrence (F-APT-13).

7. Cas limites et exemples numériques

Tous les exemples ci-dessous sont produits par le simulateur de référence et constituent des **vecteurs de test normatifs** (`annexes/vecteurs-test-recurrence.json`).

#	Cas	Entrée	Sortie attendue
E1	3 mois hebdo lundi	`BYDAY=MO`, 16/11/2026 09:00, `months=3`	14 séances, 16/11 → 15/02/2027, `until=2027-02-16`
E2	Tous les jours de départ	7 séries, `months=3`	14 séances chacune (table §3.1)
E3	Bi-hebdomadaire borné	`INTERVAL=2`, `months=3`	7 séances (16/11, 30/11, 14/12, 28/12, 11/01, 25/01, 08/02)
E4	Trois jours par semaine	`BYDAY=MO,WE,FR`, `months=3`	40 séances, dernière 15/02/2027
E5	Douze semaines	`weeks=12`	13 séances, `until=2027-02-08`
E6	Cinq occurrences bi-hebdo	`count=5`, `BYDAY=TU`, 17/11/2026	17/11, 01/12, 15/12, 29/12, 12/01/2027
E7	Date de fin incluse	`until=2026-12-14`	5 séances, la dernière le 14/12
E8	Fin de mois écrêtée	début 30/11/2026, `months=1`	5 séances, `until=2026-12-30`
E9	Mois court	`addMonths(2026-01-31, 1)`	`2026-02-28`
E10	Année bissextile	`addMonths(2024-02-29, 12)`	`2025-02-28`
E11	DST printemps (heure inexistante)	Paris, dimanche 02:30, 3 occurrences, 21/03/2027	21/03 02:30 CET, **28/03 03:00 CEST**, 04/04 02:30 CEST
E12	DST automne (heure ambiguë)	Paris, dimanche 02:30, 18/10/2026	18/10 02:30 CEST, 25/10 02:30 **CEST (premier passage)**, 01/11 02:30 CET
E13	DST Montréal	09:00, dimanche, 07/03/2027	09:00 EST (14:00Z), 09:00 EDT (13:00Z) — heure murale invariante
E14	Conflit `SKIP`	4 lundis, 23/11 occupé	16/11, 30/11, 07/12, 14/12 + `skippedConflict`
E15	Conflit `SHIFT_SAME_DAY`	16/11 09:00 occupé jusqu'à 10:00	16/11 **10:00** (+60 min), puis 09:00

#	Cas	Entrée	Sortie attendue
E16	Jour férié `SHIFT_FORWARD`	28/12 fermé	21/12, **29/12**, 04/01/2027
E17	Jour férié `SKIP`	28/12 fermé	21/12, 04/01, 11/01 + `skipped: BLACKOUT`
E18	Tampon de 15 min	séance 09:00–10:00, `post_buffer=15` , autre RDV 10:15	**pas** de conflit (empreinte = 10:15, contact autorisé)
E18b	Tampon de 15 min (conflit)	séance 09:00–10:00, `post_buffer=15` , autre RDV 10:10	**conflit** (empreinte 10:15 > 10:10) → traité par R-09
E19	Dépassement du plafond	`count=200` , offre Pro	`422 SER_007_COUNT_OVER_CAP` (`limit=104`)
E20	Fin exclusive violée	`months=3` + `count=12`	`422 SER_020_END_MODE_EXCLUSIVE`
E21	Heure trop proche	première occurrence dans 1 h	`422 SER_016_LEAD_TIME`
E22	Conflits en série	6 conflits sur 10, politique `SKIP`	`422 SER_018_CONFLICT_BUDGET` (budget 5)
E23	Trois conflits consécutifs	4 lundis, 3 premiers occupés	`422 SER_019_CONSECUTIVE_CONFLICTS`
E24	Série sans fin	`NEVER`	13 occurrences (horizon 90 j), `windowEndsAt=2027-02-08`
E25	Modification `THIS_AND_FOLLOWING`	passage bi-hebdo au 11/01/2027	occurrences avant le 11/01 conservées, suivantes recalculées, `diff` explicite
E26	Idempotence	double génération	aucune création, aucune suppression, même rapport

7.1 Détail E11 — traversée du changement d'heure (printemps)

Série : Paris, dimanche 02:30, 60 min, `count=3` , début 21/03/2027.

#	Date locale	Heure locale	Offset	UTC
1	2027-03-21	02:30	+01:00 (CET)	2027-03-21 01:30
2	2027-03-28	**03:00**	+02:00 (CEST)	2027-03-28 01:00
3	2027-04-04	02:30	+02:00 (CEST)	2027-04-04 00:30

Lecture : l'occurrence du 28/03 est avancée à 03:00 (première heure valide), reste dans la journée, et conserve **60 minutes** de durée. L'utilisateur est informé par un badge « heure ajustée (changement d'heure) » et le récapitulatif indique 03:00.

7.2 Détail E12 — ambiguïté d'automne

#	Date locale	Heure locale	Offset	UTC
1	2026-10-18	02:30	+02:00 (CEST)	2026-10-18 00:30
2	2026-10-25	02:30	**+02:00** (premier passage)	2026-10-25 00:30
3	2026-11-01	02:30	+01:00 (CET)	2026-11-01 01:30

Choix `EARLIER` documenté : on privilégie le premier passage pour éviter qu'une séance ne se retrouve après une autre dans la journée.

7.3 Cas des semaines à cheval sur deux mois

Un mois de 31 jours contient 4 ou 5 occurrences d'un même jour de semaine. Le mode `AFTER_DURATION` étant borné par **date**, il n'y a pas d'ambiguïté : « 1 mois » depuis le 30/11 donne 5 séances (30/11, 07/12, 14/12, 21/12, 28/12) alors que « 1 mois » depuis le 03/12 en donne 5 également (03, 10, 17, 24, 31/12). Si l'utilisateur veut « 4 séances par mois », il doit utiliser une autre règle (V2 : `BYMONTHDAY`).

7.4 Cas d'un client en liste d'attente sur une série

Un créneau annulé le 07/12 appartient à une série. La liste d'attente propose le créneau à un autre client ; si celui-ci accepte, l'occurrence est **détachée** et **réattribuée** (`detached` , nouveau `client_id`), et la série continue pour le client d'origine à la date suivante. Cette opération est tracée et n'est autorisée que si le praticien l'a prévue (`series_reassignable = true`).

8. Interaction avec les autres modules

Module	Interaction	Document
Disponibilités	Une occurrence hors plage disponible est un conflit `OUTSIDE_AVAILABILITY`	03 §6, 05 §4
Ressources	Réservation par occurrence, flexibilité de salle (R-19)	03 §7
Liste d'attente	Priorité sur la régénération, offre multi-candidats (R-21)	03 §16
Paiements	Trois modes de facturation (R-23)	03 §13
Notifications	Récapitulatif unique, anti-flood (R-18)	03 §12
Synchronisation	Émission d'un ICS `RRULE` unique (R-24)	03 §20
Dossier patient	Aucune note n'est créée à l'avance ; les notes sont liées à l'occurrence au moment de la séance	03 §10
Audit	Toute création/modification/suppression de série est auditée	04 §7

9. Paramétrage et administration

Paramètre	Portée	Défaut	Modifiable par
Horizon de matérialisation	Offre	90 j	Produit (plateforme)
Budget de conflits	Offre	5	Produit
Politique de conflit par défaut	Organisation	`SKIP`	Admin
Politique de jours fériés par défaut	Organisation	`SKIP`	Admin
Calendrier de jours fériés	Lieu	Pays du lieu	Admin
Tolérance `SHIFT_SAME_DAY`	Organisation	240 min	Admin
Délai minimum de réservation	Organisation	2 h	Admin
Récapitulatif de série au client	Organisation	activé	Admin
Rappel avant le début de série	Organisation	7 j	Admin
Poules de séries actives	Offre	200	Produit

Toute modification de ces paramètres est auditée et **n'affecte pas** les séries déjà créées, sauf mention explicite (« appliquer aux séries actives » avec `diff` de prévisualisation).

10. Erreurs et codes

Le catalogue complet figure au document 07 §8. Les codes propres au moteur de récurrence :

Code	HTTP	Sens
`SER_001_NO_WEEKDAY`	422	Aucun jour sélectionné
`SER_002_BAD_WEEKDAY`	422	Jour invalide ou dupliqué

Code	HTTP	Sens
`SER_003_BAD_INTERVAL`	422	Intervalle hors 1..8
`SER_004_BAD_DURATION`	422	Durée de rendez-vous invalide
`SER_005_START_WEEKDAY_MISMATCH`	422	Date de début hors des jours sélectionnés
`SER_006_BAD_COUNT`	422	Nombre d'occurrences < 1
`SER_007_COUNT_OVER_CAP`	422	Nombre d'occurrences > plafond
`SER_008_BAD_UNTIL`	422	Date de fin antérieure au début
`SER_009_BAD_DURATION_END`	422	Ni mois ni semaines fournis
`SER_010_DURATION_OVER_CAP`	422	Durée > plafond
`SER_011_INVALID_LOCAL_TIME`	422	Heure locale non résoluble
`SER_012_BAD_END_MODE`	422	Mode de fin inconnu
`SER_013_BAD_CONFLICT_POLICY`	422	Politique de conflit inconnue
`SER_014_BAD_SHIFT_POLICY`	422	Politique de jour férié inconnue
`SER_015_NEVER_WITHOUT_CAP`	422	Série sans fin sans plafond
`SER_016_LEAD_TIME`	422	Première occurrence trop proche
`SER_017_SLOT_CONFLICT`	409	Conflit bloquant (politique `FAIL`)
`SER_018_CONFLICT_BUDGET`	422	Budget de conflits dépassé
`SER_019_CONSECUTIVE_CONFLICTS`	409	Trop de conflits consécutifs
`SER_020_END_MODE_EXCLUSIVE`	422	Plusieurs modes de fin fournis
`SER_021_EXDATE_RDATE_CONFLICT`	422	Date à la fois retirée et ajoutée
`SER_022_OCCURRENCE_COLLISION`	409	Détachement vers une date déjà occupée par la série
`SER_023_TIMEZONE_REQUIRED`	422	Fuseau manquant
`SER_024_CONFIRMATION_REQUIRED`	428	Suppression sans confirmation du nombre
`SER_025_SERIES_LOCKED`	409	Série verrouillée (facturation en cours, migration)
`SER_026_PLAN_LIMIT_SERIES`	402	Plafond de séries actives de l'offre atteint

11. Critères d'acceptation du module

#	Critère	Test
CA-1	« 3 mois, 1 fois par semaine » produit 14 séances, dernière le 15/02/2027, `until=2027-02-16`	T-REC-001
CA-2	Les 7 jours de départ produisent 14 séances (table §3.1)	T-REC-002
CA-3	Une occurrence tombant une heure inexistante est décalée à la première heure valide et reste dans la journée	T-REC-014
CA-4	Une occurrence ambiguë utilise le premier passage	T-REC-015
CA-5	Aucune occurrence silencieusement perdue : chaque absence a un motif	T-REC-020
CA-6	Une double génération ne crée ni doublon ni suppression	T-REC-026b
CA-7	`FAIL` sur conflit ne crée **aucune** occurrence	T-REC-030
CA-8	`SHIFT_SAME_DAY` ne sort jamais du jour et respecte la tolérance	T-REC-031
CA-9	Un jour férié en `SHIFT_FORWARD` est reporté au premier jour disponible, sans doublon	T-REC-021

#	Critère	Test
CA-10	Le détachement survit à une régénération	T-REC-029
CA-11	La suppression `THIS_AND_FOLLOWING` conserve l'historique et les factures	T-REC-040 et T-REC-045
CA-12	Le plafond de l'offre est appliqué avec un message actionnable	T-REC-048
CA-13	L'export ICS se réimporte à l'identique dans Google, Outlook et Apple	T-REC-057 à T-REC-059
CA-14	Le simulateur de référence et l'implémentation de production donnent le même résultat sur 10 000 tirages	T-REC-054

12. Traçabilité

Exigence	Règles	Documents
F-REC-01	R-01	03 §4, 06 §5
F-REC-02	R-02	03 §4, 06 §5
F-REC-03, F-REC-04	R-03, R-04, R-05	03 §4, 06 §3
F-REC-05	R-14	07 §6
F-REC-06	R-09	06 §5, 07 §6
F-REC-07	R-07	06 §5
F-REC-08	R-08	06 §5
F-REC-09	R-13	06 §5, 07 §7
F-REC-10	R-10, R-16	06 §5, 06 §9
F-REC-11	R-10, R-09	06 §5
F-REC-12	R-18	06 §5
F-REC-13	R-12	06 §5, 07 §7
F-REC-14	R-22	06 §5, 07 §7
F-REC-15	R-11, R-17	06 §5, 06 §6
F-REC-16	R-15	07 §5 (états)
F-REC-17	R-14	06 §5, 07 §6
F-REC-18	R-06, R-20	06 §5
F-REC-19	R-24	06 §5
F-REC-20	R-12, R-13	06 §5, 07 §7

13. Questions ouvertes (à arbitrer avant la fin du lot 1)

#	Question	Proposition	Échéance
Q1	Faut-il permettre `interval` en jours (tous les 10 jours) ?	Non en V1 (fréquence hebdomadaire uniquement) ; V2 si la demande est > 5 % des séries	Lot 1
Q2	La borne de fin doit-elle être incluse ou exclue par défaut ?	Incluse, avec prévisualisation explicite (D2)	Lot 1
Q3	Faut-il autoriser 7 jours (`BYDAY` complet) ?	Non : refuser encourage l'usage de la récurrence quotidienne prévue en V2	Lot 1
Q4	Récapitulatif de série : un message par séance ou un seul ?	Un seul (anti-flood), avec fichier ICS joint	Lot 2

#	Question	Proposition	Échéance
Q5	Que faire d'une occurrence future lorsqu'un praticien part ?	Transfert proposé, jamais silencieux (R-22)	Lot 3
Q6	Facturation par défaut : `PER_SESSION` ou forfait ?	`PER_SESSION` , l'utilisateur choisit le forfait	Lot 3
Q7	Faut-il un mode « série en attente de validation » (approbation client) ?	Oui, en V1.1 (`series.status = PENDING_APPROVAL`)	V1.1
Q8	Support des séries partagées entre deux praticiens (binôme) ?	Non en V1, V2 (`practitioner_ids[]`)	V2

14. Validation de cette spécification

```
# 1. Invariants du moteur (400 séries aléatoires, 5 fuseaux, DST incluses)
python3 tools/recurrence_simulator.py --self-test

# 2. Exemples normatifs (§3.1, E1, E6)
python3 tools/recurrence_simulator.py --demo

# 3. Régénération des vecteurs de test
python3 tools/recurrence_simulator.py --vectors > annexes/vecteurs-test-recurrence.json

# 4. Validation du schéma JSON (17 cas)
python3 tools/test_schemas.py
```

Résultat attendu à la date de publication : `OK — 400 séries aléatoires, 6 invariants vérifiés.` et `Schéma conforme à Draft 2020-12 — 17 cas vérifiés.`

Le test local porte sur **400 séries** pour rester rapide ; l'exécution de CI décrite au document 09 §1.2 monte à **10 000 séries** et compare les deux implémentations (Python et TypeScript) — toute divergence est bloquante.

Suite : `07-api-et-schemas-rdv-recurrents.md` *— mise en œuvre exposée (endpoints, machines à états, erreurs).*

Document 07

API et schémas

Document : 07 / 11 · **Version** 1.0 · **Date** : 12 novembre 2026 · **Statut** : contrat **Base d'URL** : `https://api.myrdv.app/v1` · **Format** : JSON (UTF-8), dates ISO-8601 **Authentification** : OAuth 2.0 (authorization code + PKCE) ou clé d'API d'organisation **Normativité** : les règles de comportement renvoient au document 06 (préfixes `R-xx`). Schéma de validation : `schemas/recurrence-rule.schema.json`.

1. Conventions

Sujet	Convention
Versionnage	Préfixe de chemin `/v1` ; les évolutions compatibles n'incrémentent pas la version ; les ruptures introduisent `/v2` avec 12 mois de support
Identifiants	ULID (26 caractères, triables) — jamais d'entier séquentiel exposé
Dates	`YYYY-MM-DD` pour les dates locales ; `HH:MM` pour les heures murales ; RFC 3339 avec offset pour les instants
Fuseaux	Nom IANA obligatoire (`Europe/Paris`) — jamais d'abréviation (`CET` , `EST`)
Pagination	`?limit=50&cursor=<ulid>` ; réponse `{data, nextCursor, hasMore}`
Tri	`?sort=startAt,-createdAt`
Idempotence	En-tête `Idempotency-Key` (UUID v4) obligatoire sur tous les `POST` / `PATCH` / `DELETE` mutants ; conservation 24 h
Concurrence optimiste	En-tête `If-Match: <etag>` sur les `PATCH` ; `412 Precondition Failed` sinon
Erreurs	Enveloppe unique (§8), codes stables, `requestId` pour la corrélation support
Traçabilité	En-têtes `X-Request-Id` (client) et `X-Correlation-Id` (propagé aux webhooks)
Langue	`Accept-Language: fr-FR` ; les messages d'erreur sont localisés, le code reste stable
Débit	600 requêtes/min par organisation, 60/min par utilisateur pour les routes `write` ; en-têtes `RateLimit-*`
Champs inconnus	Refusés (`additionalProperties: false`) — protège contre les fautes de frappe silencieuses
Unités	Durées en minutes (entier), montants en centimes + devise ISO-4217
Sécurité des réponses	Aucune donnée de santé dans les payloads de série (pas de motif, pas de diagnostic)

2. Ressources et endpoints

> **Note**
> **Périmètre.** L'API publique V1 compte **62 endpoints**. Ce document en spécifie **46** : l'intégralité de ceux qui touchent aux séries, aux occurrences et aux ressources nécessaires pour les intégrer. Les 16 endpoints restants (administration de l'organisation, facturation avancée, téléconsultation, facturation d'abonnement) sont décrits dans la référence OpenAPI 3.1 publiée.

2.1 Séries (14 endpoints)

Méthode	Chemin	Rôle requis	Description
`POST`	`/series`	`series:write`	Crée une série et matérialise les occurrences (R-11)

Méthode	Chemin	Rôle requis	Description
POST	/series/preview	series:write	Prévisualisation (dryRun), aucune écriture (F-REC-05)
GET	/series	series:read	Liste filtrée (practitionerId , clientId , status , from , to)
GET	/series/{id}	series:read	Détail + état + progression
PATCH	/series/{id}	series:write	Modifie la règle avec scope (THIS / THIS_AND_FOLLOWING / ALL)
POST	/series/{id}/regenerate	series:regenerate	Régénère à horizon (idempotent, R-17)
POST	/series/{id}/pause	series:write	Passe en PAUSED (aucune nouvelle occurrence)
POST	/series/{id}/resume	series:write	Reprise, régénération à l'horizon courant
DELETE	/series/{id}	series:delete	Annulation/suppression avec scope et confirmCount (R-22)
GET	/series/{id}/occurrences	series:read	Occurrences, paginées, avec statut et detached
GET	/series/{id}/report	series:read	Rapport de dernière génération (R-14)
GET	/series/{id}/revisions	series:read	Historique des règles (30 j)
POST	/series/{id}/revert	series:write	Retour à une révision (R-12)
GET	/series/{id}/export.ics	series:read	Export ICS d'un VEVENT RRULE (R-24)

2.2 Occurrences (5 endpoints)

Méthode	Chemin	Rôle	Description
PATCH	/occurrences/{key}	appointment:write	Détache et modifie une occurrence (R-13)
DELETE	/occurrences/{key}	appointment:delete	Retire une occurrence (ajoute un exdate)
POST	/occurrences/{key}/rdate	appointment:write	Réintègre une occurrence retirée ou ajoute une date
GET	/occurrences/{key}	appointment:read	Détail
POST	/occurrences/{key}/confirm	appointment:write	Confirmation de présence

2.3 Aides à la décision et ressources d'intégration (27 endpoints)

Méthode	Chemin	Description
POST	/series/suggest-slots	Propose des créneaux récurrents libres (même jour/heure, N occurrences)
POST	/series/validate	Valide une règle sans générer (contrôle des plafonds, R-16)
GET	/series/quota	Consommation des plafonds de l'offre
GET	/schedules	Semaines types de disponibilité (practitionerId requis)
GET	/appointments	Rendez-vous isolés (recherche from / to / status)
POST	/waitlist	Inscription en liste d'attente (F-WAIT-01)
POST	/notifications/preview	Prévisualisation d'un modèle avec variables
GET	/audit	Journal d'audit filtré (actorId , subjectId , période)
GET	/clients	Recherche de clients (q , pagination)
GET	/clients/{id}	Fiche client et consentements
POST	/clients	Création manuelle d'une fiche

Méthode	Chemin	Description
`GET`	`/practitioners`	Praticiens de l'organisation et leurs services
`GET`	`/services`	Catalogue des services (durée, tarif, tampons)
`GET`	`/locations`	Lieux, salles et équipements disponibles
`GET`	`/availability`	Créneaux disponibles (`practitionerId` , `serviceId` , `from` , `to`)
`GET`	`/invoices`	Factures d'une période (`clientId` , `status`)
`POST`	`/invoices`	Émission d'une facture
`POST`	`/payments/intent`	Intention de paiement Stripe
`GET`	`/reports/activity`	Rapport d'activité agrégé
`GET`	`/reports/no-show`	Taux de présence et coût des absences
`POST`	`/webhooks`	Enregistrement d'un point de terminaison
`GET`	`/webhooks/{id}/deliveries`	Historique et rejeu des livraisons
`POST`	`/exports`	Demande d'export asynchrone (JSON/CSV/PDF)
`GET`	`/exports/{id}`	État et lien de téléchargement
`POST`	`/auth/token`	Échange de code OAuth 2.0
`POST`	`/auth/refresh`	Rotation du refresh token
`GET`	`/me`	Profil, organisations, permissions effectives

3. Créer une série

3.1 Requête

```
POST /v1/series HTTP/1.1
Host: api.myrdv.app
Authorization: Bearer <token>
Idempotency-Key: 6f1b6f4e-6a1e-4f5b-9d1a-2c0a3b7c9d10
Content-Type: application/json
```

```
{
  "practitionerId": "01JB7Q9F3K2W8M5V6X1Y4Z7A0B",
  "clientId": "01JB7QA1M4N6P8R2S5T9U3V7W1",
  "serviceId": "01JB7QB2X8Y4Z6A1B3C5D7E9F2",
  "locationId": "01JB7QC4G7H2J5K8L1M4N6P9Q3",
  "resourceIds": ["01JB7QD5R8T1V4W7X0Y3Z6A9B2"],
  "rule": {
    "freq": "WEEKLY",
    "interval": 1,
    "byWeekday": [1],
    "startDate": "2026-11-16",
    "startTime": "09:00",
    "timezone": "Europe/Paris",
    "durationMinutes": 60,
    "end": { "mode": "AFTER_DURATION", "months": 3 },
    "advanced": {
      "conflictPolicy": "SKIP",
      "dateShiftPolicy": "SHIFT_FORWARD",
      "skipHolidays": true,
      "notifyClient": true
    }
  },
  "billing": { "mode": "PER_SESSION" },
  "notifyClient": true
}
```

3.2 Réponse 201 Created


```
{
  "id": "01JB7QE6S9U2W5X8Y1Z4A7B0C3",
  "status": "ACTIVE",
  "practitionerId": "01JB7Q9F3K2W8M5V6X1Y4Z7A0B",
  "clientId": "01JB7QA1M4N6P8R2S5T9U3V7W1",
  "serviceId": "01JB7QB2X8Y4Z6A1B3C5D7E9F2",
  "rule": {
    "freq": "WEEKLY",
    "interval": 1,
    "byWeekday": [1],
    "startDate": "2026-11-16",
    "startTime": "09:00",
    "timezone": "Europe/Paris",
    "durationMinutes": 60,
    "end": { "mode": "AFTER_DURATION", "months": 3 },
    "advanced": { "conflictPolicy": "SKIP", "dateShiftPolicy": "SHIFT_FORWARD" }
  },
  "effectiveUntil": "2027-02-16",
  "progress": { "planned": 14, "materialized": 14, "completed": 0, "cancelled": 0 },
  "requested": 14,
  "count": 14,
  "materializedUntil": "2027-02-15",
  "skipped": [],
  "skippedConflict": [],
  "warnings": [],
  "createdAt": "2026-11-12T10:22:31+01:00",
  "etag": "\"7c9f2a\"",
  "links": {
    "self": "/v1/series/01JB7QE6S9U2W5X8Y1Z4A7B0C3",
    "occurrences": "/v1/series/01JB7QE6S9U2W5X8Y1Z4A7B0C3/occurrences",
    "ics": "/v1/series/01JB7QE6S9U2W5X8Y1Z4A7B0C3/export.ics"
  }
}
```


3.3 Code de réponse et sémantique

Cas	HTTP	Corps
Création complète	`201`	Série + occurrences (R-14 respecté)
Création partielle (`SKIP` a retiré des occurrences)	`201`	Identique, `status: "PARTIAL"` , `skipped[]` non vide
Conflit bloquant (`FAIL`)	`409`	`SER_017_SLOT_CONFLICT` avec `details.dates`
Plafond d'offre	`422`	`SER_007_COUNT_OVER_CAP` / `SER_010_DURATION_OVER_CAP` avec `details.upgradeUrl`
Règle invalide	`422`	Le code le plus spécifique (§8)
Idempotence rejouée	`200`	Le corps du premier appel, `Idempotency-Replayed: true`

4. Prévisualiser (dryRun)

`POST /v1/series/preview` accepte le même corps que `POST /v1/series` (sans `billing`), **n'écrit rien**, et renvoie le rapport complet. Il est **obligatoirement** appelé par l'interface avant toute création lorsque le nombre d'occurrences dépasse 12 (F-REC-05).


```
{
  "rule": { "...": "..." },
  "dryRun": true
}
```

```
{
  "effectiveUntil": "2027-02-16",
  "count": 12,
  "requested": 14,
  "status": "PARTIAL",
  "occurrences": [
    { "key": "01JB7QE6S9U2W5X8Y1Z4A7B0C3@20261116T0900", "date": "2026-11-16", "startTime": "09:00", "endTime": "10:00" },
    { "key": "01JB7QE6S9U2W5X8Y1Z4A7B0C3@20261123T0900", "date": "2026-11-23", "startTime": "09:00", "endTime": "10:00" }
  ],
  "skipped": [
    { "date": "2026-12-25", "reason": "BLACKOUT", "policy": "SKIP" },
    { "date": "2027-01-01", "reason": "BLACKOUT", "policy": "SKIP" }
  ],
  "skippedConflict": [],
  "warnings": ["2 séances non planifiées sur 14 demandées (jours fériés)."],
  "estimated": { "durationMs": 42, "conflictChecks": 14 }
}
```


Note de volumétrie : `occurrences` est limité à 400 entrées ; au-delà, `occurrences` est tronqué et `occurrencesTruncated: true` est renvoyé avec `count` exact.

5. Machines à états

5.1 États de la série

TRANSITIONS AUTORISÉES

Départ	→	Pending Approval	création avec approbation requise
	→	Active	création directe
Pending Approval	→	Active	validation du praticien
	→	Rejected	refus
Active	→	Paused	pause (aucune nouvelle matérialisation)
	→	Ended	fin de règle atteinte (borne de fin)
	→	Exhausted	plafond d'occurrences atteint
	→	Cancelled	suppression
Paused	→	Active	reprise + régénération
	→	Cancelled	suppression

État	Matérialisation	Modifiable	Supprimable	Visible agenda
`PENDING_APPROVAL`	Non	Oui	Oui	Non (sauf praticien)
`ACTIVE`	Oui	Oui	Oui	Oui
`PAUSED`	Non (existantes conservées)	Oui	Oui	Oui
`ENDED`	Non	Non (sauf réouverture V2)	Oui	Oui (historique)
`EXHAUSTED`	Non	Oui (`count` extensible sous plafond)	Oui	Oui
`CANCELLED`	Non	Non	—	Partiel (traces)
`REJECTED`	Non	Non	Oui	Non

Transitions interdites : `CANCELLED → ACTIVE` (il faut créer une nouvelle série, pour préserver la lisibilité de l'audit), `ENDED → ACTIVE` en V1, `EXHAUSTED → PAUSED`.

5.2 États de l'occurrence

TRANSITIONS AUTORISÉES

Départ	→	Held	réservation en ligne (15 min)
	→	Scheduled	création praticien
Held	→	Scheduled	vérification du client
	→	Expired	expiration
Scheduled	→	Confirmed	confirmation du client (SMS/e-mail)
	→	Arrived	accueil
	→	No Show	
	→	Cancelled By Client	
	→	Cancelled By Provider	
	→	Rescheduled	
Confirmed	→	Arrived	
	→	No Show	
	→	Cancelled By Client	
	→	Cancelled By Provider	
Arrived	→	Completed	
Completed	→	Invoiced	
No Show	→	Invoiced	

État	Facturable	Libère le créneau	Compte dans la progression
`HELD`	Non	Non (réservé)	Non
`SCHEDULED`	Après exécution	Oui si annulé	Oui
`CONFIRMED`	Après exécution	Oui si annulé	Oui
`ARRIVED`	Oui (à la clôture)	Non	Oui
`COMPLETED`	Oui	Non	Oui (terminée)
`NO_SHOW`	Selon politique	Oui	Oui
`CANCELLED_*`	Selon délai	Oui	Oui (annulée)
`RESCHEDULED`	Non pour cette occurrence	Oui	Oui (remplacée)
`INVOICED`	Déjà facturée	Non	Oui
`EXPIRED`	Non	Oui	Non

5.3 Progression exposée

`GET /series/{id}` renvoie toujours :

```
"progress": {
  "planned": 14,        // occurrences planifiées par le moteur jusqu'à la borne
  "materialized": 14,   // présentes en base à l'instant t
  "future": 12,         // date > maintenant
  "completed": 1,
  "cancelled": 1,
  "skipped": 0,
  "label": "2 / 14 effectuées"
}
```

L'interface affiche `label` ; le champ `label` est localisé selon `Accept-Language` .

6. Politique de conflit dans l'API

Valeur	Effet	Réponse si non satisfaisable
`FAIL`	Transaction annulée si une occurrence pose problème	`409 SER_017_SLOT_CONFLICT`, `details: {dates: [...]}`
`SKIP`	Occurrences en conflit omises, listées	`201` avec `skippedConflict[]`
`SHIFT_SAME_DAY`	Décalage intra-journée, pas de 15 min	`409` si `shiftToleranceMinutes` insuffisant

Attention contractuelle : `SKIP` **ne renvoie pas** `409` — c'est une réussite partielle, ce qui doit être vérifié par le client d'API. Le champ `status` (`COMPLETE` / `PARTIAL`) permet de décider.

7. Modifier une série

7.1 Portées

`scope`	`from`	Effet	Réponse
`THIS`	—	Détache l'occurrence ciblée (`occurrenceKey` obligatoire)	`200` , `seriesRevisionId` inchangé
`THIS_AND_FOLLOWING`	date	Fractionne la série : ancienne règle tronquée, nouvelle série liée (`parentSeriesId`)	`200` avec `diff`
`ALL`	—	Modifie la règle et régénère les futures non détachées	`200` avec `diff`

7.2 Requête ALL

```
PATCH /v1/series/01JB7QE6S9U2W5X8Y1Z4A7B0C3
If-Match: "7c9f2a"
Idempotency-Key: 0c2f5a9e-11b0-4f7e-9b3c-1e5d8a7f6c21
```

```
{
  "scope": "ALL",
  "rule": { "startTime": "09:30", "end": { "mode": "AFTER_COUNT", "count": 12 } },
  "notifyClients": true
}
```

7.3 Réponse avec diff obligatoire

```
{
  "id": "01JB7QE6S9U2W5X8Y1Z4A7B0C3",
  "seriesRevisionId": "01JB7QF7T0V3X6Y9Z2A5B8C1D4",
  "diff": {
    "created": ["2027-02-15T09:30"],
    "updated": ["2026-12-07T09:30", "2026-12-14T09:30"],
    "deleted": ["2027-02-01T09:00", "2027-02-08T09:00"],
    "detachedPreserved": ["2026-11-30T09:00"],
    "pastUntouched": 6
  },
  "count": 12,
  "status": "COMPLETE",
  "revertibleUntil": "2026-12-12T10:22:31+01:00"
}
```

Règles. Aucune occurrence passée n'apparaît dans `deleted` ; les occurrences détachées apparaissent dans `detachedPreserved` ; le client d'API doit présenter ce `diff` à l'utilisateur avant d'appliquer si `scope = ALL` et `diff.deleted.length > 0` (paramètre `dryRun: true` disponible).

7.4 Suppression

```
DELETE /v1/series/01JB7QE6S9U2W5X8Y1Z4A7B0C3?scope=THIS_AND_FOLLOWING&from=2027-01-11&confirmCount=5
```

Situation	Réponse
`confirmCount` ≠ nombre réel	`428 SER_024_CONFIRMATION_REQUIRED` avec `details.expectedCount`
Occurrences facturées dans le périmètre	`200`, les facturées passent à `CANCELLED` (jamais supprimées)
Série verrouillée	`409 SER_025_SERIES_LOCKED`

8. Catalogue d'erreurs

Enveloppe unique :

```
{
  "error": {
    "code": "SER_007_COUNT_OVER_CAP",
    "message": "Votre offre autorise au maximum 104 occurrences par série.",
    "details": { "limit": 104, "requested": 200, "plan": "pro", "upgradeUrl": "https://app.myrdv.app/billing" },
    "requestId": "req_01JB7QG8V1W4Y7Z0A3B6C9D2E5",
    "documentation": "https://docs.myrdv.app/errors/SER_007_COUNT_OVER_CAP"
  }
}
```

Code	HTTP	Catégorie	Action utilisateur
`SER_001_NO_WEEKDAY`	422	Validation	Choisir au moins un jour
`SER_002_BAD_WEEKDAY`	422	Validation	Corriger le jour
`SER_003_BAD_INTERVAL`	422	Validation	Intervalle 1..8
`SER_004_BAD_DURATION`	422	Validation	Durée 5..480
`SER_005_START_WEEKDAY_MISMATCH`	422	Validation	Aligner la date de début
`SER_006_BAD_COUNT`	422	Validation	Nombre ≥ 1
`SER_007_COUNT_OVER_CAP`	422	Plafond	Réduire ou changer d'offre
`SER_008_BAD_UNTIL`	422	Validation	Date de fin ≥ début
`SER_009_BAD_DURATION_END`	422	Validation	Fournir mois ou semaines
`SER_010_DURATION_OVER_CAP`	422	Plafond	Réduire la durée
`SER_011_INVALID_LOCAL_TIME`	422	Validation	Choisir une heure valide
`SER_012_BAD_END_MODE`	422	Validation	Mode inconnu
`SER_013_BAD_CONFLICT_POLICY`	422	Validation	Politique inconnue
`SER_014_BAD_SHIFT_POLICY`	422	Validation	Politique inconnue
`SER_015_NEVER_WITHOUT_CAP`	422	Validation	Fournir un plafond
`SER_016_LEAD_TIME`	422	Règle métier	Décaler le début (> 2 h)
`SER_017_SLOT_CONFLICT`	409	Conflit	Choisir un autre créneau / politique `SKIP`
`SER_018_CONFLICT_BUDGET`	422	Conflit	Revoir la règle
`SER_019_CONSECUTIVE_CONFLICTS`	409	Conflit	Vérifier fermeture/congés
`SER_020_END_MODE_EXCLUSIVE`	422	Validation	Un seul mode de fin
`SER_021_EXDATE_RDATE_CONFLICT`	422	Validation	Retirer la date d'une des deux listes
`SER_022_OCCURRENCE_COLLISION`	409	Conflit	Choisir une autre date

Code	HTTP	Catégorie	Action utilisateur
`SER_023_TIMEZONE_REQUIRED`	422	Validation	Fournir `timezone`
`SER_024_CONFIRMATION_REQUIRED`	428	Sécurité	Fournir `confirmCount`
`SER_025_SERIES_LOCKED`	409	État	Attendre la fin du verrou
`SER_026_PLAN_LIMIT_SERIES`	402	Plafond	Archiver des séries ou changer d'offre
`AUTH_401_TOKEN_EXPIRED`	401	Auth	Rafraîchir le jeton
`AUTH_403_FORBIDDEN_SCOPE`	403	Auth	Permission manquante
`VAL_400_SCHEMA`	400	Validation	Corriger le corps
`RATE_429_LIMIT`	429	Débit	Attendre `Retry-After`
`IDEM_409_KEY_REUSE`	409	Idempotence	Clé réutilisée avec un corps différent

9. Webhooks

Événement	Déclencheur	Charge utile utile
`series.created`	Création	`seriesId` , `count` , `effectiveUntil`
`series.updated`	`PATCH`	`seriesId` , `scope` , `diff`
`series.ended`	Fin de règle	`seriesId` , `reason`
`series.exhausted`	Plafond atteint	`seriesId` , `count`
`series.partial`	Génération partielle	`seriesId` , `skipped[]` (R-14)
`occurrence.detached`	Détachement	`key` , `before` , `after`
`occurrence.cancelled`	Annulation	`key` , `reason` , `by`
`occurrence.confirmed`	Confirmation client	`key` , `confirmedAt`
`occurrence.completed`	Fin de séance	`key` , `invoiceId?`

Signature : `X-MyRDV-Signature: t=<timestamp>,v1=<hmac_sha256(t + "." + body)>`, fenêtre de tolérance 5 minutes, secret par endpoint, rotation semestrielle. **Rejeu** : 3 tentatives exponentielles (1 min, 5 min, 30 min), puis mise en file morte consultable. **Idempotence** : `X-MyRDV-Event-Id` stable ; le récepteur doit dédupliquer.

10. Exemples d'intégration

10.1 Créer une série bornée à 3 mois (cURL)

```
curl -X POST https://api.myrdv.app/v1/series \
  -H "Authorization: Bearer $TOKEN" \
  -H "Idempotency-Key: $(uuidgen)" \
  -H "Content-Type: application/json" \
  -d '{
    "practitionerId": "01JB7Q9F3K2W8M5V6X1Y4Z7A0B",
    "clientId": "01JB7QA1M4N6P8R2S5T9U3V7W1",
    "serviceId": "01JB7QB2X8Y4Z6A1B3C5D7E9F2",
    "rule": {
      "freq": "WEEKLY", "interval": 1, "byWeekday": [1],
      "startDate": "2026-11-16", "startTime": "09:00", "timezone": "Europe/Paris",
      "end": { "mode": "AFTER_DURATION", "months": 3 }
    }
  }'
```

10.2 Décaler une seule occurrence (Node.js)

```
const key = encodeURIComponent("01JB7QE6S9U2W5X8Y1Z4A7B0C3@20261207T0900");

await fetch(`https://api.myrdv.app/v1/occurrences/${key}`, {
  method: "PATCH",
  headers: {
    Authorization: `Bearer ${token}`,
    "Content-Type": "application/json",
    "Idempotency-Key": crypto.randomUUID(),
    "If-Match": seriesEtag
  },
  body: JSON.stringify({ startTime: "14:00", durationMinutes: 60, reason: "Indisponibilité" })
});
// => 200 { detached: true, notifyClient: true }
```

10.3 Gérer une création partielle (Python)

```
resp = session.post(f"{BASE}/series", json=payload, headers={"Idempotency-Key": str(uuid4())})
if resp.status_code == 201:
    data = resp.json()
    if data["status"] == "PARTIAL":
        # R-14 : aucune perte silencieuse — afficher le détail
        for item in data["skipped"]:
            print(f"Non planifiée : {item['date']} ({item['reason']})")
elif resp.status_code == 409 and resp.json()["error"]["code"] == "SER_017_SLOT_CONFLICT":
    dates = resp.json()["error"]["details"]["dates"]
    print("Conflits :", dates)
```

10.4 Rejouer une requête en toute sécurité

Toute création rejouée avec la même `Idempotency-Key` et le même corps renvoie `200` avec `Idempotency-Replayed: true` et **ne crée pas** de seconde série. Une clé réutilisée avec un corps différent renvoie `409 IDEM_409_KEY_REUSE`.

11. Atlas des champs de la ressource Series

Champ	Type	Écriture	Description
`id`	ULID	lecture	Identifiant
`status`	enum	lecture (sauf pause/resume)	`PENDING_APPROVAL`, `ACTIVE`, `PAUSED`, `ENDED`, `EXHAUSTED`, `CANCELLED`, `REJECTED`
`practitionerId`	ULID	création	Praticien
`clientId`	ULID	création	Client
`serviceId`	ULID	création	Service (durée, tarif, tampons)
`locationId`	ULID	création	Lieu (fuseau par défaut)
`resourceIds`	ULID[]	création	Salles/équipements (R-19)
`rule`	objet	`PATCH`	Règle (schéma JSON)
`effectiveUntil`	date	lecture	Borne calculée (R-04/R-05)
`materializedUntil`	date	lecture	Dernière occurrence matérialisée (R-11)
`progress`	objet	lecture	Compteurs (§5.3)
`skipped` / `skippedConflict`	tableau	lecture	Rapport R-14
`billing.mode`	enum	création	`PER_SESSION`, `PREPAID_PACKAGE`, `SUBSCRIPTION` (R-23)

Champ	Type	Écriture	Description
`parentSeriesId`	ULID?	lecture	Renseigné après un fractionnement (R-12)
`detachedCount`	entier	lecture	Nombre d'occurrences détachées
`createdBy` / `updatedBy`	ULID	lecture	Audit
`etag`	chaîne	lecture	Concurrence optimiste

12. Conformité du contrat

Exigence	Vérification
Schéma JSON valide (Draft 2020-12)	`python3 tools/test_schemas.py` — 17 cas
Aucun endpoint ne renvoie de donnée de santé	revue de contrat + test automatisé d'absence de champs cliniques (doc 09 T-SEC-012)
Idempotence sur tous les mutants	test T-API-030
Erreurs stables et documentées	génération de la page `/errors` depuis le catalogue §8
Pagination sur toutes les collections	test T-API-040
Aucune fuite d'identifiant d'un autre tenant	test T-SEC-020 (isolation multi-tenant)

Contrat exécutable : `api/openapi.yaml` *· documentation GUI :* `api/index.html` *· règles transverses et conformité :* `11-api-regles-et-conformite.md`.